

Lekce 08: Funkce

Úvod do Pythonu na Micro:bitu

Proč funkce?

Opakující se kód je problém:

```
# bez funkce -- stejný kód 3x  
if button_a.was_pressed():  
    display.show(Image.HAPPY)  
    sleep(500)  
    display.clear()  
if button_b.was_pressed():  
    display.show(Image.HAPPY)  
    sleep(500)  
    display.clear()
```

Proč funkce?

Opakující se kód je problém:

```
# bez funkce -- stejný kód 3x  
if button_a.was_pressed():  
    display.show(Image.HAPPY)  
    sleep(500)  
    display.clear()  
if button_b.was_pressed():  
    display.show(Image.HAPPY)  
    sleep(500)  
    display.clear()
```

DRY — Don't Repeat Yourself

Opakující se kód dáme do **funkce**. Funkci pak voláme **kdekoliv, kolikrát chceme** — kód napíšeme jen jednou.

Definice funkce — def

```
def blikni():  
    display.show(Image.HAPPY)  
    sleep(500)  
    display.clear()
```

- ▶ def — klíčové slovo „definuj funkci“
- ▶ blikni — název funkce (pravidla jako pro proměnné)
- ▶ () — závorky jsou povinné (i bez parametrů)
- ▶ : a odsazení — stejně jako u if nebo while

Definice funkce — def

```
def blikni():  
    display.show(Image.HAPPY)  
    sleep(500)  
    display.clear()
```

- ▶ def — klíčové slovo „definuj funkci“
- ▶ blikni — název funkce (pravidla jako pro proměnné)
- ▶ () — závorky jsou povinné (i bez parametrů)
- ▶ : a odsazení — stejně jako u if nebo while

Definice vs. volání

def blikni(): funkci **připraví**, ale **nespustí**.

blikni() funkci **spustí** — závorky jsou nutné i při volání!

Parametry a argumenty

Parametr = proměnná v definici; **argument** = hodnota při volání:

```
def pozdrav(jmeno):           # jmeno = parametr
    display.scroll("Ahoj, " + jmeno + "!")

pozdrav("Karel")              # "Karel" = argument
pozdrav("Jana")
```

Parametr

Lokální proměnná funkce.
Existuje jen uvnitř.
Definujeme ji v závorce.

Argument

Konkrétní hodnota při volání.
Přiřadí se do parametru.
Píšeme do závorek při volání.

return — návratová hodnota

Funkce může **vrátit** výsledek zpět volajícímu:

```
import random

def hod_kostkou(pocet_sten):
    return random.randint(1, pocet_sten)

cislo = hod_kostkou(6)          # cislo = vysledek
display.show(str(cislo))
```

- ▶ return ukončí funkci a vrátí hodnotu
- ▶ Kód za return se **neprovede**
- ▶ Funkce bez return vrátí None

return — návratová hodnota

Funkce může **vrátit** výsledek zpět volajícímu:

```
import random

def hod_kostkou(pocet_sten):
    return random.randint(1, pocet_sten)

cislo = hod_kostkou(6)          # cislo = vysledek
display.show(str(cislo))
```

- ▶ return ukončí funkci a vrátí hodnotu
- ▶ Kód za return se **neprovede**
- ▶ Funkce bez return vrátí None

Tip

Funkce s return je jako kalkulačka — zadáš vstup, dostaneš výsledek.

Funkce bez return je jako tiskárna — provede akci, nic nevrátí.

Teploměr z lekce 06 — přepíšeme podmínky do funkce:

```
from microbit import *

def zobraz_stav(teplota):
    if teplota > 27:
        display.show(Image.HAPPY)
    elif teplota < 18:
        display.show(Image.SAD)
    else:
        display.show(Image.MEH)

while True:
    zobraz_stav(temperature())
    sleep(2000)
```

Hlavní smyčka je nyní **přehledná** — stačí jeden řádek.

Funkci `zobraz_stav` lze snadno znovu použít nebo upravit.

- ✓ `def nazev():` — definuje funkci
- ✓ Funkce se spustí až při **volání** `nazev()`
- ✓ **Parametr** — proměnná v definici; **argument** — hodnota při volání
- ✓ `return hodnota` — funkce vrátí výsledek
- ✓ Funkce bez `return` vrací `None`
- ✓ **Refaktoring** — přepis kódu do funkcí pro přehlednost (DRY)